CO2-AT3-Debugging and Optimization

CSA-0603-DESIGN OF ANALYSIS AND ALGORITHMS

Shaik Basheer

Register Number: 192465049

Q33. Debugging Linear Search Code (Incorrect Index Variable Update)

The following Linear Search implementation incorrectly updates the result index even after finding the target, causing the function to return the last occurrence instead of the first one. Carefully analyze the logic and identify why the intended search behavior is violated. Apply systematic debugging to ensure that the first valid occurrence is returned immediately when found. Optimize the implementation for readability and correct control flow. Analyze the time complexity of the corrected solution and rewrite the final code with clear comments.

```
def linear_search(arr, key):
pos = -1
for i in range(len(arr)):
if arr[i] == key:
pos = i
return pos
```

Aim:

To identify and correct the error in the given Linear Search program where the result index is incorrectly updated after finding the target, debug the algorithm to return the **first occurrence** of the key, improve the code for readability and correct control flow, and analyze its time complexity.

Algorithm:

1. Read the input array and the search key.
2. Start from the first element of the array.
3. Initialize the result index pos as -1.
4. Compare each element with the search key.
5. If the current element matches the key, store its index in pos.
6. Return pos immediately after finding the first matching element.

7. Continue until the key is found or all elements are checked.
8. Return -1 if the key is not present in the array.

Given Incorrect Code

```
def linear_search(arr, key):
    pos = -1
    for i in range(len(arr)):
        if arr[i] == key:
            pos = i
    return pos
```

Errors Identified

Error 1: Incorrect Index Update

```
if arr[i] == key:
    pos = i
```

Problem:

- The index pos is updated every time the key is found.
- The loop continues even after finding the first occurrence.
- If the key appears multiple times, pos is overwritten with the index of the later occurrence.
- Therefore, the function returns the last occurrence instead of the first occurrence.

Correct Statement

```
if arr[i] == key:
    return i
```

Error 2: Missing Immediate Termination

Problem:

- The search does not stop after finding the first matching element.
- Continuing the loop is unnecessary when the required first occurrence has already been found.
- Returning the index immediately improves the control flow and avoids unnecessary comparisons.

Correct Statement

if arr[i] == key:

    return i

Corrected Program:

main.py

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter search key: "))
position = linear_search(arr, key)
print("Element found at index:", position)
```

input

```
Enter elements: 10 20 30 40
Enter search key: 20
Element found at index: 1


...Program finished with exit code 0
Press ENTER to exit console.
```

Sample Input

Enter elements: 10 20 30 20 40

Enter search key: 20

Sample Output

Element found at index: 1

Step-by-Step Debugging

Initial Array

[10, 20, 30, 20, 40]

Search Key = 20

Step 1

- Compare 10 with 20
- 10 != 20
- Continue searching.

Index = 0

Step 2

- Compare 20 with 20
- 20 == 20 → Match found.
- Return index 1 immediately.

Index = 1

Final Result

Element found at index: 1

The search stops at the **first occurrence** of 20, even though the same value appears again at index 3.

Optimizations

❖ Changed pos = i to return i so that the first valid occurrence is returned immediately.

❖ Removed the unnecessary pos variable because the index can be returned directly when the key is found.

❖ Added meaningful comments to improve code readability and understanding.

❖ Used proper indentation and formatting to follow Python coding standards.

❖ Used descriptive variable names such as arr, key, and i for better code clarity.

❖ Removed the logical error that caused the function to return the last occurrence.

❖ Added immediate termination after finding the first occurrence.

❖ Avoided unnecessary comparisons after the target element is found.

❖ Maintained a simple linear search approach without using additional data structures.

Time Complexity Analysis

Case	Time Complexity
Best Case (First element matches)	O(1)
Average Case	O(n)
Worst Case (Last element matches / Not found)	O(n)

Explanation

- **Best Case – O(1):** The key is found at the first position, so only one comparison is required.
- **Average Case – O(n):** The key is generally found somewhere in the middle of the array.
- **Worst Case – O(n):** The key is at the last position or is not present, so all elements must be checked.

Space Complexity

O(1)

The algorithm uses only a constant amount of extra space and does not require any additional data structure.

Result

The error in the Linear Search program was successfully identified and corrected. The algorithm now returns the **first occurrence of the search key immediately** when it is found, instead of continuing the search and returning the last occurrence. The optimized version provides clearer control flow, improved readability, and runs in **O(1) time in the best case and O(n) time in the average and worst cases**, using **O(1) extra space**.